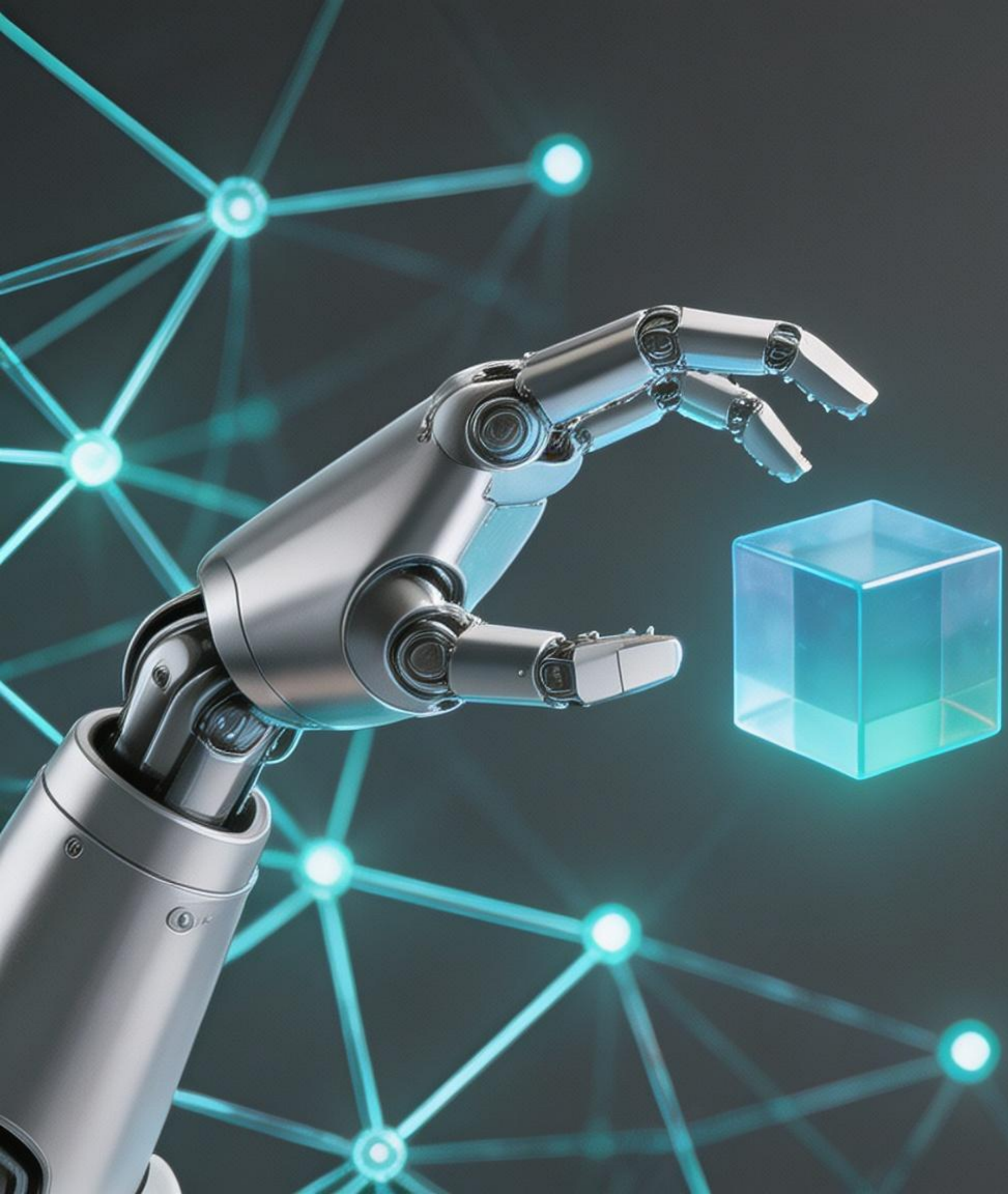




Fundamentals of Robot Learning

A Comprehensive Guide

Presenter: Mengyuan Wang

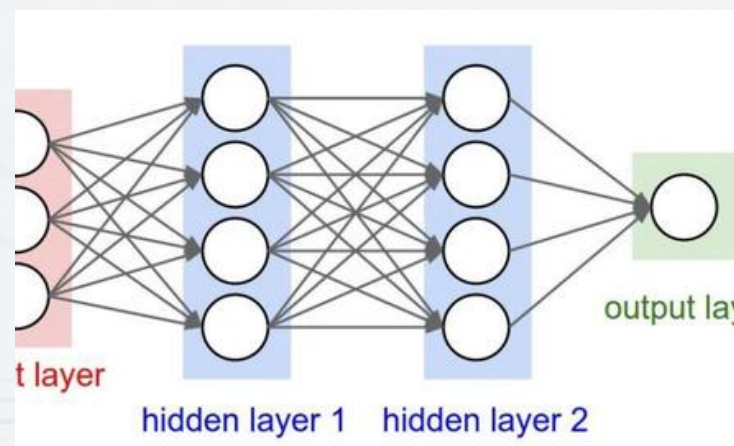
Date: 2026/08/15

Table of Contents

01	The Difference: Robot Learning vs. Ordinary Machine Learning	05	Main Learning Paradigms: RL vs. IL
02	Closed-Loop Sequence: From Observation to Action	06	Behavior Cloning (BC)
03	Core Concepts: Trajectory, Reward, Demonstration	07	Action Chunking
04	Robot Degrees of Freedom, Action Space, and Control	08	ACT: Action Chunking with Transformers

1. Robot Learning vs. Ordinary Machine Learning

Ordinary Machine Learning (OML)



Focuses on **static pattern recognition** from pre-collected, curated datasets. It operates within a closed, digital realm with no real-world physical interaction.

-  **Static, Passive Data:** Uses independent, identically distributed (i.i.d.) datasets with no temporal dependency.
-  **Open-Loop Process:** Predictions do not influence the input data; a one-way information flow.
-  **Abundant Data:** Relies on large, readily available digital datasets with low cost of collection.

Robot Learning (RL)



Physically embodied and environmentally situated: Emphasizes learning by doing through direct interaction with a dynamic physical environment to achieve a long-term goal.

-  **Interactive, Sequential Data:** Data is generated by the robot's actions, creating a causal loop between decision and outcome.
-  **Closed-Loop Dynamics:** Actions alter the environment, which in turn provides new observations for the next step.
-  **Safety-Critical & Efficient:** High stakes for failure.

2. Closed-Loop Sequence: From Observation to Action

Observation (O)

Decision (Policy)

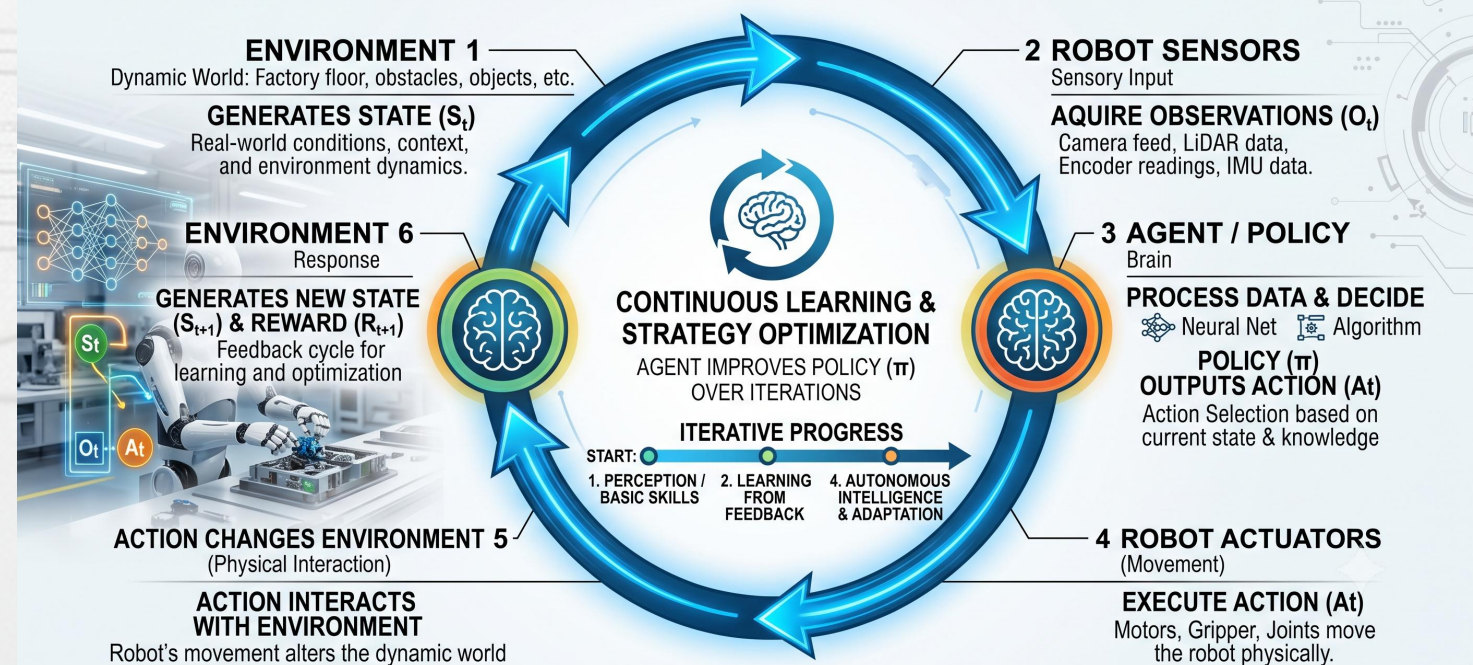
Action (A)

Environment Change

New Observation

CLOSED-LOOP ITERATIVE LEARNING PROCESS FOR ROBOTIC INTELLIGENCE EVOLUTION

From Perception to Autonomous Intelligence through Continuous Practice and Optimization



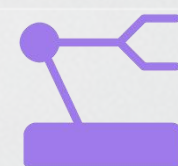
Closed-Loop Iterative Learning Framework for Robotic Intelligence Evolution

State (S_t)— The complete, objective configuration of the environment, which is often hidden and unobservable directly.

Observation (O_t)— Partial, sensor-derived data (e.g., camera feeds) that is often noisy and incomplete.

Action(a_t)— Action represents the specific control commands executed by a robot, such as joint targets or velocities.

Policy (π_θ)— The robot's "brain": a mapping function that translates observations into optimal actions.



$$a_t = \pi_\theta(o_t)$$

$$a_t = \pi_\theta(o_{1:t})$$

$$a_t = \pi_\theta(o_{t-H:t})$$

2.1 The Critical Difference: State vs. Observation



Observation $\neq$ State



State (s_t) — The Ground Truth

The complete, objective reality of the environment — every physical detail, force, and entity in perfect, unfiltered clarity. This is the **omniscient perspective** that no real-world system can ever measure directly.

Example: In self-driving, this means knowing the exact speed, acceleration, and intent of every car, pedestrian, and cyclist within 1 km — including those hidden around corners or obscured by other vehicles.

💡 **Inaccessible in reality, existing only in theory or simulation.**

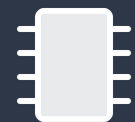


Observation (o_t) — The Robot's View

The imperfect, sensor-derived snapshot of the world. Observations are inherently noisy, limited by hardware resolution, and spatially incomplete — **missing information hidden from the robot's field of view.**

Example: Blurry camera images with lens flare, a sparse LiDAR point cloud missing small objects, or delayed GPS data. A pedestrian blocked by a truck would not appear in the robot's observations at all.

⚠️ **The sole basis for decision-making, and the primary source of uncertainty.**



The Core Challenge: Robots can never access the "true state." They must learn to make optimal decisions based solely on flawed, incomplete observations. Bridging this gap is the fundamental problem of perception and control in robotics.

2.2 Observation



Hardware Foundation: The physical basis for perception and action, and the first line of defense against observation challenges.

01 Sensing Modalities

- **Visual Perception:** RGB provides semantics and texture, while RGB-D and point clouds construct 3D geometry.
- **Proprioception:** Joint angles and torque feedback form the foundation of motion control.
- **Force/Tactile Sensing:** critical for delicate tasks such as assembly and compliant grasping.

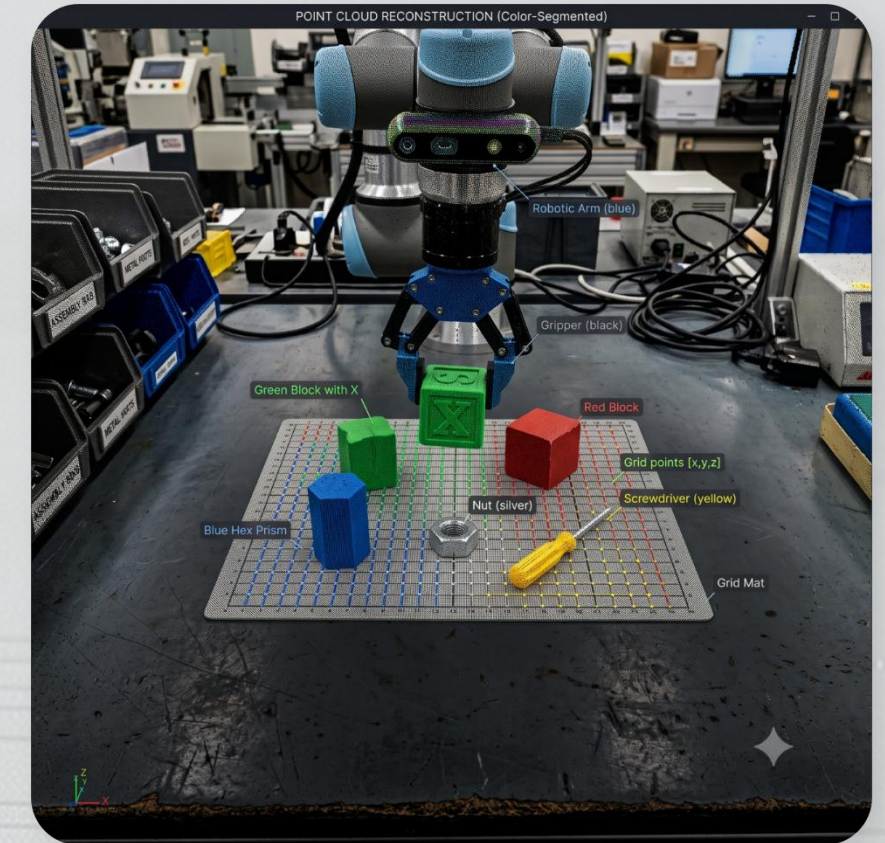
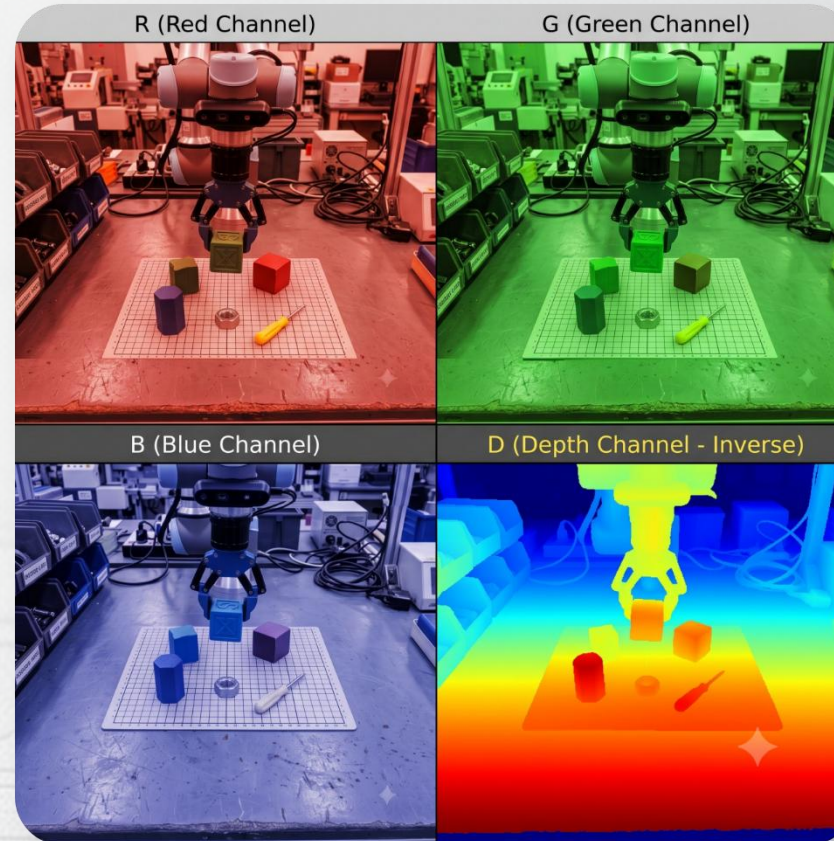
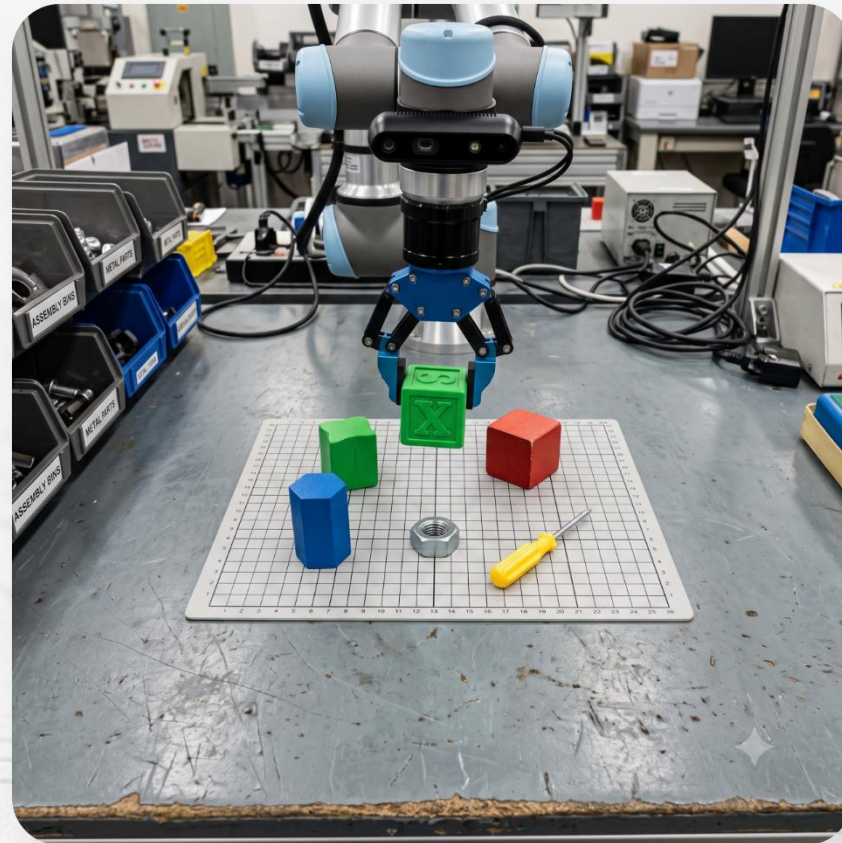
02 Key Challenges

- Partially Observable
- Occlusion
- Noise
- Latency
- Multi-view Fusion
- Coordinate Transformation
- History Dependence

03 POMDP

Real-world robotic tasks are generally better modeled as POMDPs rather than fully observable MDPs.

2.2.1 Visual Perception: RGB, RGB-D, and Point Clouds



01 RGB

Advantages: Low cost, rich in color and texture details, and highly compatible with mature pretrained vision models for semantic understanding.

Limitations: Merely a 2D projection lacking 3D depth information, and highly sensitive to environmental factors such as lighting changes and object occlusion.

02 RGB-D

Advantages: Simultaneously captures appearance and geometric depth, providing critical 3D spatial references for autonomous navigation, dynamic obstacle avoidance, and object grasping.

Limitations: Inherent noise in depth data, limited measurement accuracy for transparent or highly reflective materials, and a relatively restricted effective detection range.

03 Point Clouds

Advantages: Directly represents the geometric structure of 3D space with the richest detail, making it an ideal input for high-precision mapping and complex manipulator operations.

Limitations: Massive unstructured data leads to extremely high computational and storage costs, along with high algorithmic complexity for data denoising, registration, and feature extraction.

2.2.2 Proprioception

The perception of a robot's own state is the core foundation for precise motion control, autonomous localization, and intelligent decision-making.



01 Joint Angles

Provides real-time feedback on the current absolute or relative position of each servo joint.



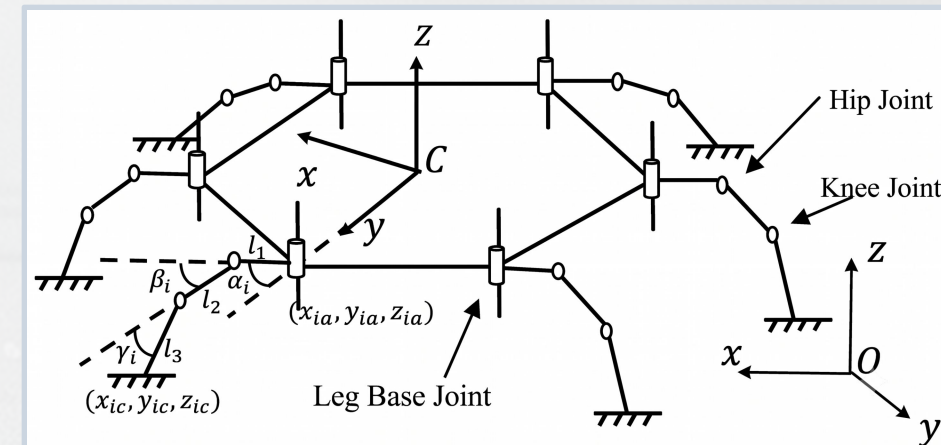
02 Joint Velocity

Monitors changes in the angular velocity of joint rotation, enabling smooth trajectory planning.



03 Joint Torque

Senses the force and torque output at the joints to ensure operational safety.



Multi-Joint Coordinated Perception

Through high-precision sensing of each joint, the robot can construct a digital "skeleton map" of its own body.



04 End-Effector Pose

Determines the precise coordinates and orientation of the manipulator's end-effector in 3D space.



05 Gripper State Perception

Provides real-time feedback on the opening/closing degree and gripping force.



Core Value: The Foundation of Autonomous Learning for Robots — Without proprioceptive state perception, a robot would lose control over its own limbs, rendering it incapable of any autonomous decision-making or delicate manipulation.

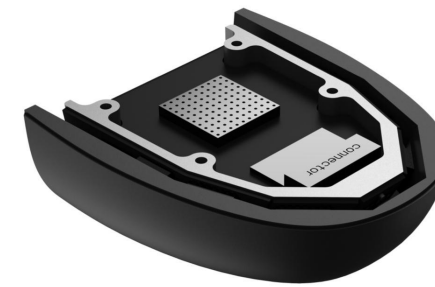
2.2.3 Force/Tactile Sensing

In delicate tasks such as assembly and polishing, visual information alone is often insufficient to determine contact status and force conditions.



01 Force/Torque

Core Function: Real-time measurement of interaction forces and torques between the robot and the environment, providing precise force feedback to prevent excessive or insufficient force.



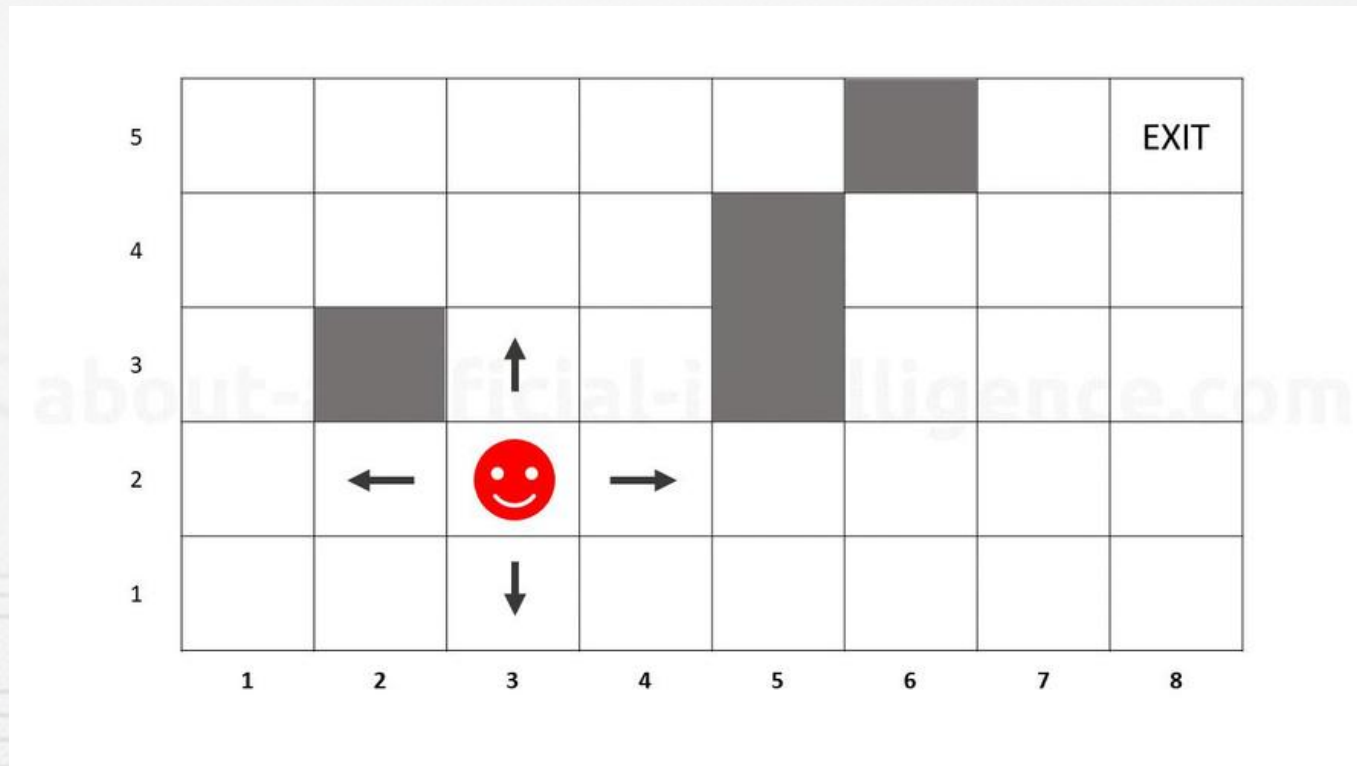
02 Tactile

Core Function: Perception of pressure distribution, micro-deformations, and object slippage trends at the contact area, and even recognition of surface textures and materials.



Core Insight: Combining vision and tactile sensing allows the robot not only to "see" but also to "feel and precisely control," marking a crucial step toward intelligent compliant manipulation.

3. Core Concepts: Trajectory, Reward, Demonstration



This grid world example illustrates how an agent navigates an environment. Every step involves a **Policy** decision, contributing to a **Trajectory**, guided by the **Reward** signal towards the goal, or learned from expert **Demonstrations**.

Trajectory/episode/rollout (τ)

The complete story of an episode: $\tau = (o_1, a_1, o_2, a_2, \dots, o_T, a_T)$. Each path has an associated return $G(\tau)$, which is the cumulative sum of rewards obtained along the path.

Reward ($r_t = r(o_t, a_t)$)

A scalar value evaluating the desirability of an action. Designing a good reward function is critical but often challenging, especially with sparse or deceptive rewards.

Demonstration ($\mathcal{D} = \{\tau_i\}_{i=1}^N$)

A dataset collected from an expert performing the task, allowing the agent to learn by mimicking successful behavior.

4.1 What is Degrees of Freedom (DoF)?

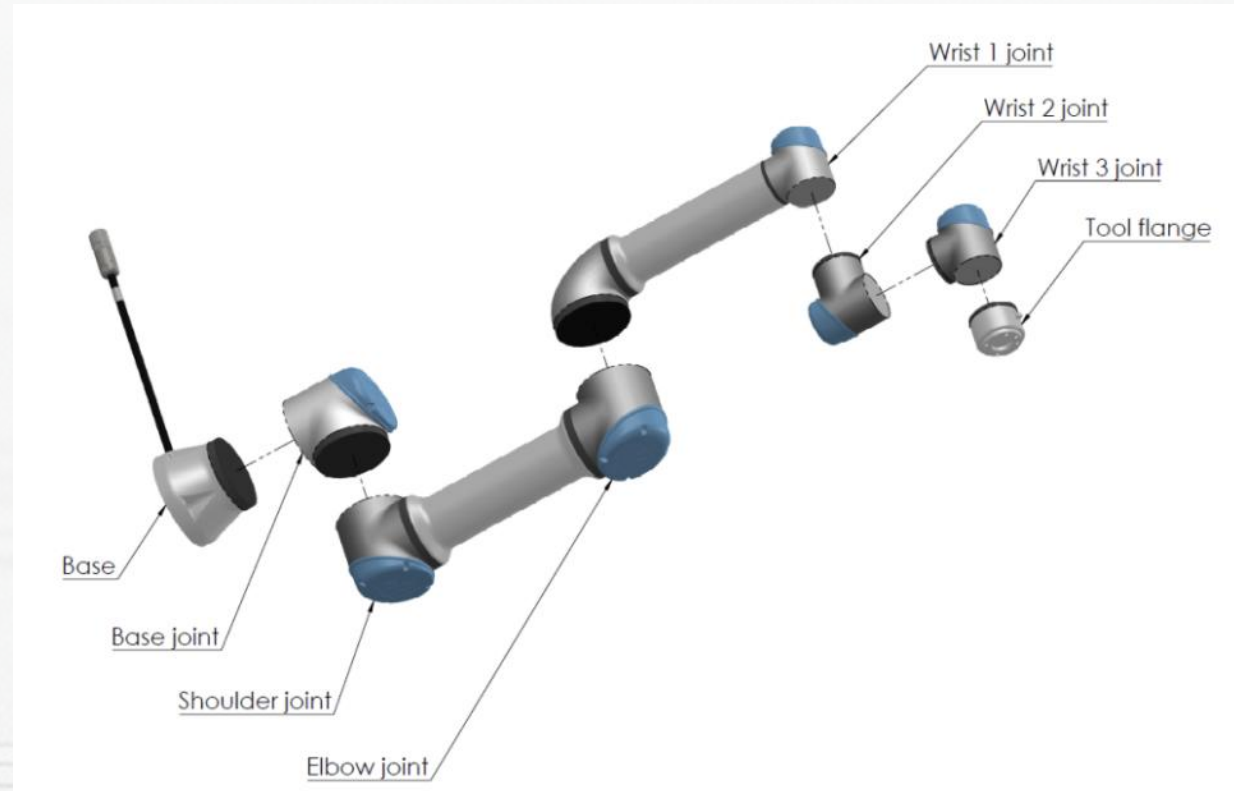


Figure: UR5e Robot Arm – Joints, Base, and Tool Flange

Definition

The count of independent motion parameters defining a robot's state.

Intuitive Insight: Dimensions from 1 to 6

A simple hinge joint has only 1 DoF; whereas a free rigid body possesses the full 6 DoF in 3D space, comprising 3 translational degrees of freedom along the X, Y, and Z axes, and 3 rotational degrees of freedom around the three axes (roll, pitch, and yaw).

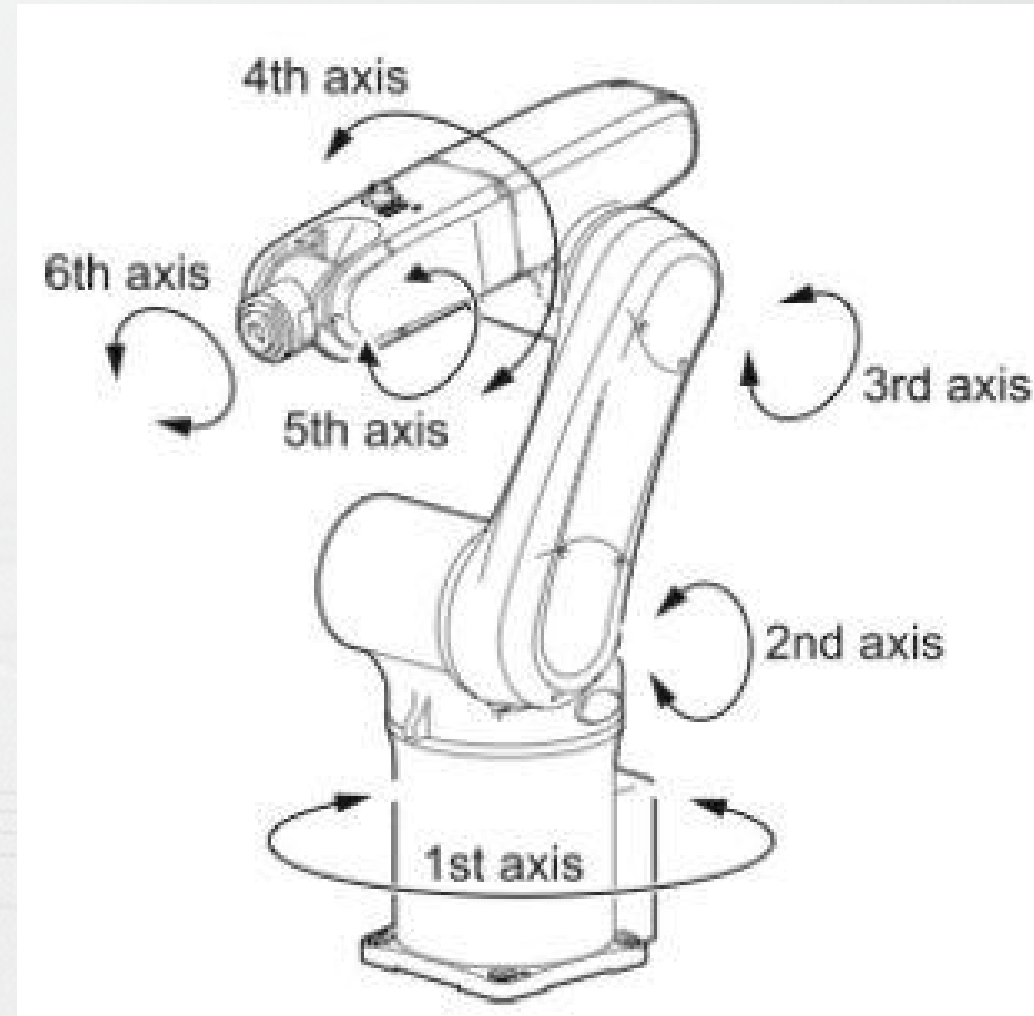


Figure: A standard 6-axis articulated robotic arm, highlighting its rotational degrees of freedom which enable precise manipulation in 3D space.

Applications and Classification in Robotics

Joint Degrees of Freedom: In a standard serial manipulator, each independently driven single-DoF joint contributes one actuated degree of freedom to the system.

End-Effector Degrees of Freedom: Refers to the pose (6D Pose) that the end-effector can achieve in space, which directly determines the robot's workspace and dexterity.

4.2 Redundant DoF, Underactuation, and High-Dimensional Control



Figure: A typical collaborative operation scenario of multi-DoF industrial manipulators.

01 Redundant DoF

- Redundancy: **Joint DoF > Task DoF** (e.g., 7-DoF arm for 6-DoF end-effector).
- Advantages: Enhanced obstacle avoidance & reachability.
- Limitations: Complex inverse kinematics & control design.

02 Underactuation

- Underactuation: **Actuators < DoF** (Common in humanoids & UAVs).
- Core: Exploit dynamic coupling for indirect control of unactuated DoFs.

03 Challenges and Opportunities in High-Dimensional Control

- Challenge: **DoF ↑ → State Space Expands Exponentially** (High compute & data demand).
- Opportunity: Richer locomotion & better adaptability.

4.3 Action Space

01 Discrete Action Space

High-Level Decision (Multiple-Choice):

- ✓ Action: Select from discrete options (e.g., pick, place).
- ✓ Algorithm: Classification.

02 Continuous Action Space

Low-Level Control (Fill-in-the-Blanks):

- ✓ Action: Output continuous values ($a_t \in \mathbb{R}^{d_a}$) for joints/torque.
- ✓ Algorithm: Regression.

Engineering Insight:

Core: Continuous actions dominate for high-precision & high-dynamic control.

Discrete: High-level "trigger signals" only.

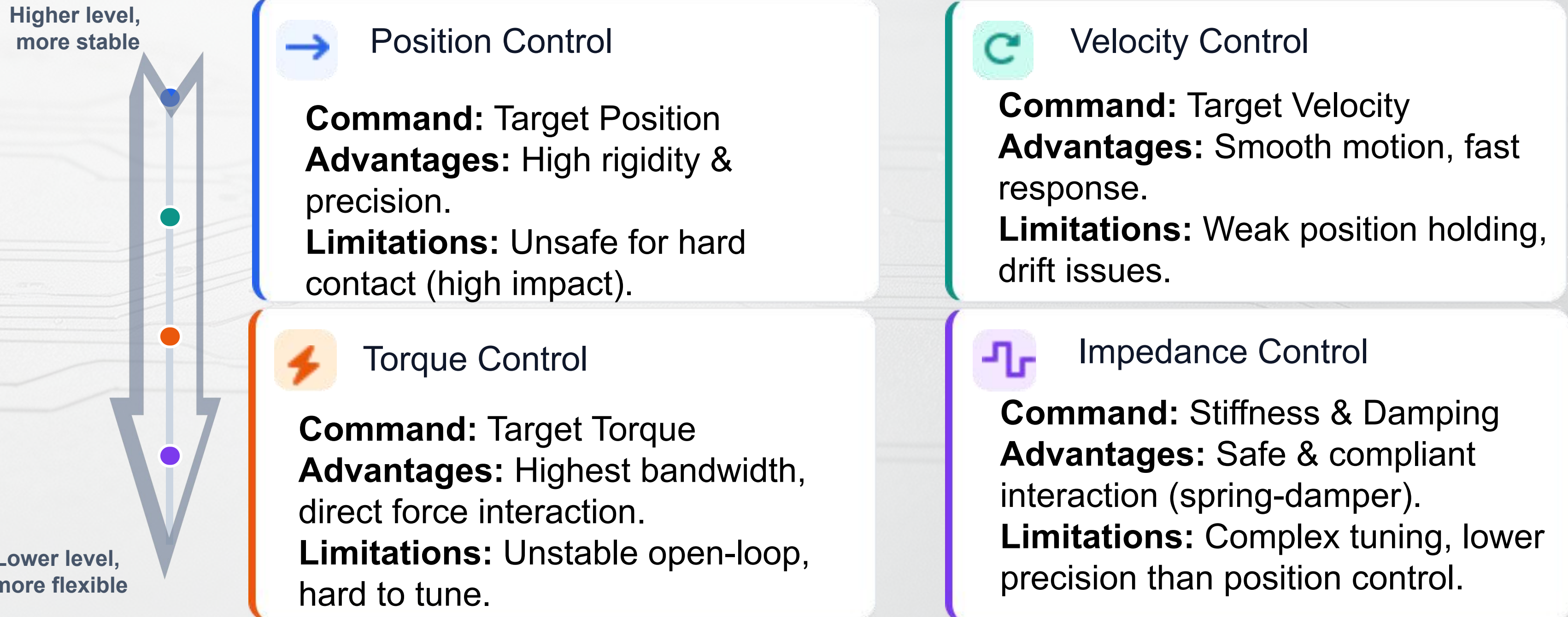
Continuous: Low-level execution relies on continuous numerical commands.

4.4 Joint Space Actions vs. Cartesian Space Actions

Dimension	Cartesian Space/Task Space	Joint Space
Focus	End-Effector / Task Target	Robot Arm Joints / Motors
Representation	Position (X, Y, Z) + Orientation (Roll, Pitch, Yaw) $a_t = [\Delta x, \Delta y, \Delta z, \Delta r_x, \Delta r_y, \Delta r_z, g]$	7DOF: $a_t = [q_1, q_2, \dots, q_7, g]$
Dimension Fixedness	Fixed at 6D (governed by the physical laws of 3D space)	Depends on the arm's structure (equal to the number of joints)
Intuitiveness	Highly intuitive for humans (e.g., "Move the gripper forward by 10 cm")	Extremely non-intuitive for humans (e.g., "Rotate Axis 1 by 10° , Axis 2 by -5° ")
Control Complexity	Requires complex mathematical transformations and suffers from singularity issues	Most direct for motor actuation

4.5 Control Modes: Position, Velocity, Torque, and Impedance

The same learned policy behaves differently under different robot control interfaces.



Takeaway: Position and velocity are common in imitation learning; torque is lower level; impedance bridges learned policies and real contact.

5. Main Learning Paradigms: RL vs. IL



Figure: Expert demonstrations via VR/AR teleoperation enable efficient skill transfer to the robotic arm.

01. Reinforcement Learning (RL) – Trial-and-Error

- ◆ Core: Optimize behavior via environment rewards $\max_{\theta} \mathbb{E}_{\pi_{\theta}} [\sum_{t=1}^T \gamma^{t-1} r_t]$.
- ◆ Advantages: Discovers novel, super-human strategies.
- ◆ Limitations: Sample-inefficient, credit assignment & exploration-exploitation dilemmas.

02. Imitation Learning (IL) – Learning by Demonstration

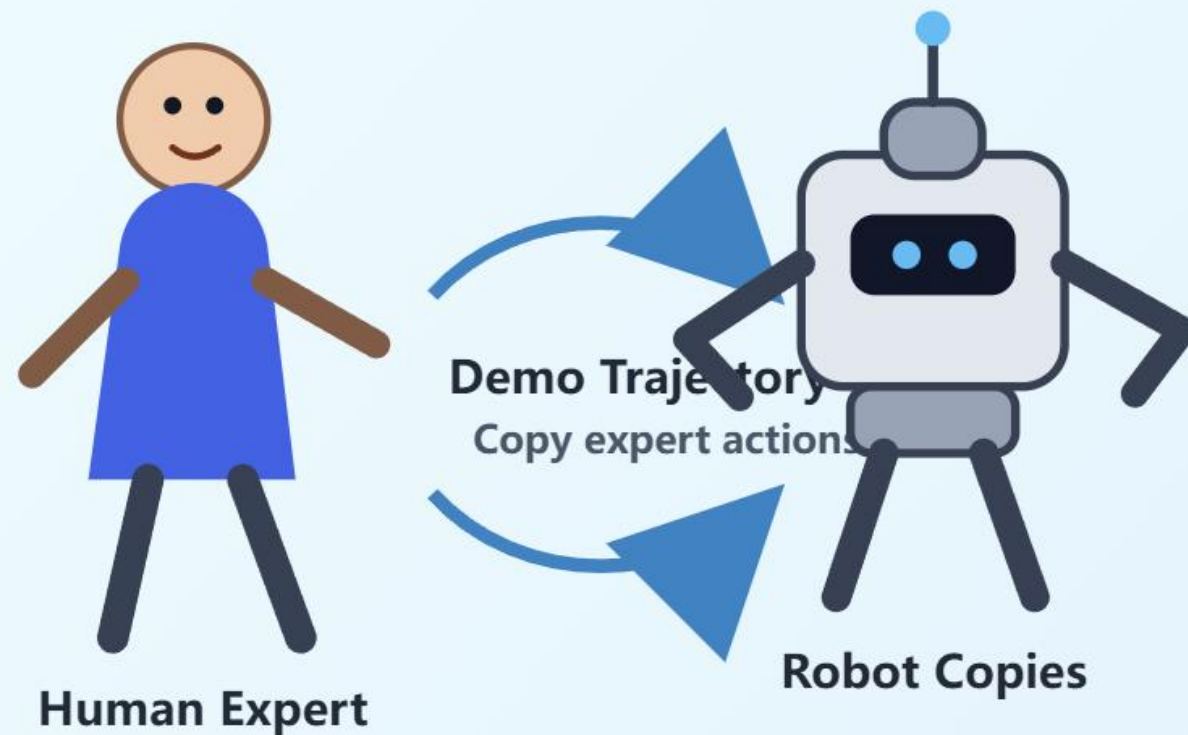
- ◆ Core: Mimic expert behavior.
- ◆ Advantages: Sample-efficient, no complex reward design.
- ◆ Limitations: Limited by expert, error compounding & covariate shift.

5. Main Learning Paradigms: RL vs. IL

Imitation Learning

Learn by watching expert demonstrations

Expert Data

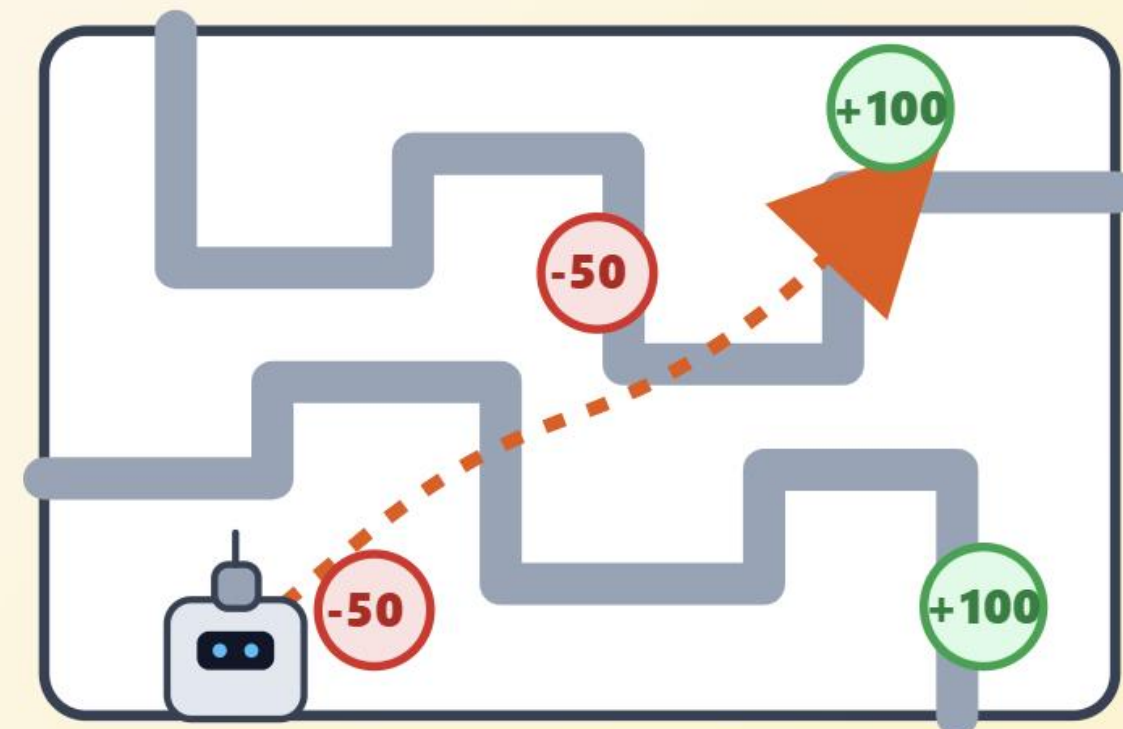


Expert actions → Supervised learning → Policy

Reinforcement Learning

Learn by trial-and-error to maximize return

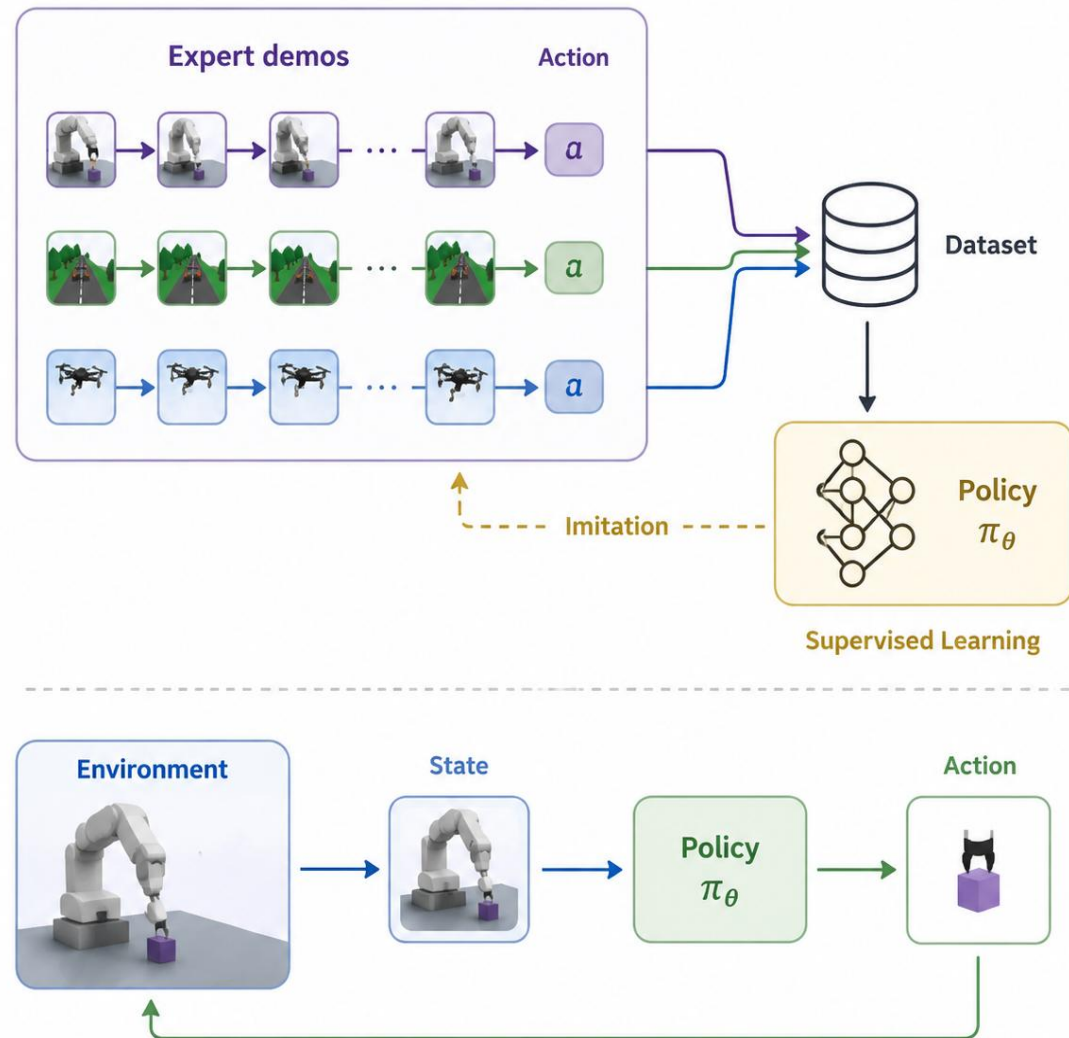
Environment Feedback



Action → Feedback → Policy update

6. Behavior Cloning (BC)

Behavior Cloning



Behavior Cloning (BC):

- Paradigm: Supervised Learning
- Mechanism: Direct observation-to-Action Mapping
- Key Feature: No Reward Signals Required

Core Mechanism: A 3-Step Process

01. Data Capture

$$\mathcal{D} = \{(o_t, a_t)\}_{t=1}^N$$

02. Supervised Training

$$a_t \approx \pi_\theta(o_t)$$

Continuous Action Space:

$$\min_{\theta} \sum_t \|\pi_\theta(o_t) - a_t\|^2$$

Discrete Action Space:

$$\min_{\theta} - \sum_t \log \pi_\theta(a_t | o_t)$$

03. Policy Execution

The trained model directly outputs actions $\pi_\theta(o_t)$ for the agent.

⚠ Catastrophic Compound Error

Small deviations accumulate, causing the agent to drift into unobserved states where the policy fails entirely.

📊 Extremely Data-Hungry

Requires massive, diverse datasets to cover all edge cases, making it impractical for complex real-world tasks.

📊 Performance Ceiling

The model can never outperform the expert data it was trained on, limiting its potential for autonomous improvement.

6.1 Core Issues in BC: Distribution Shift

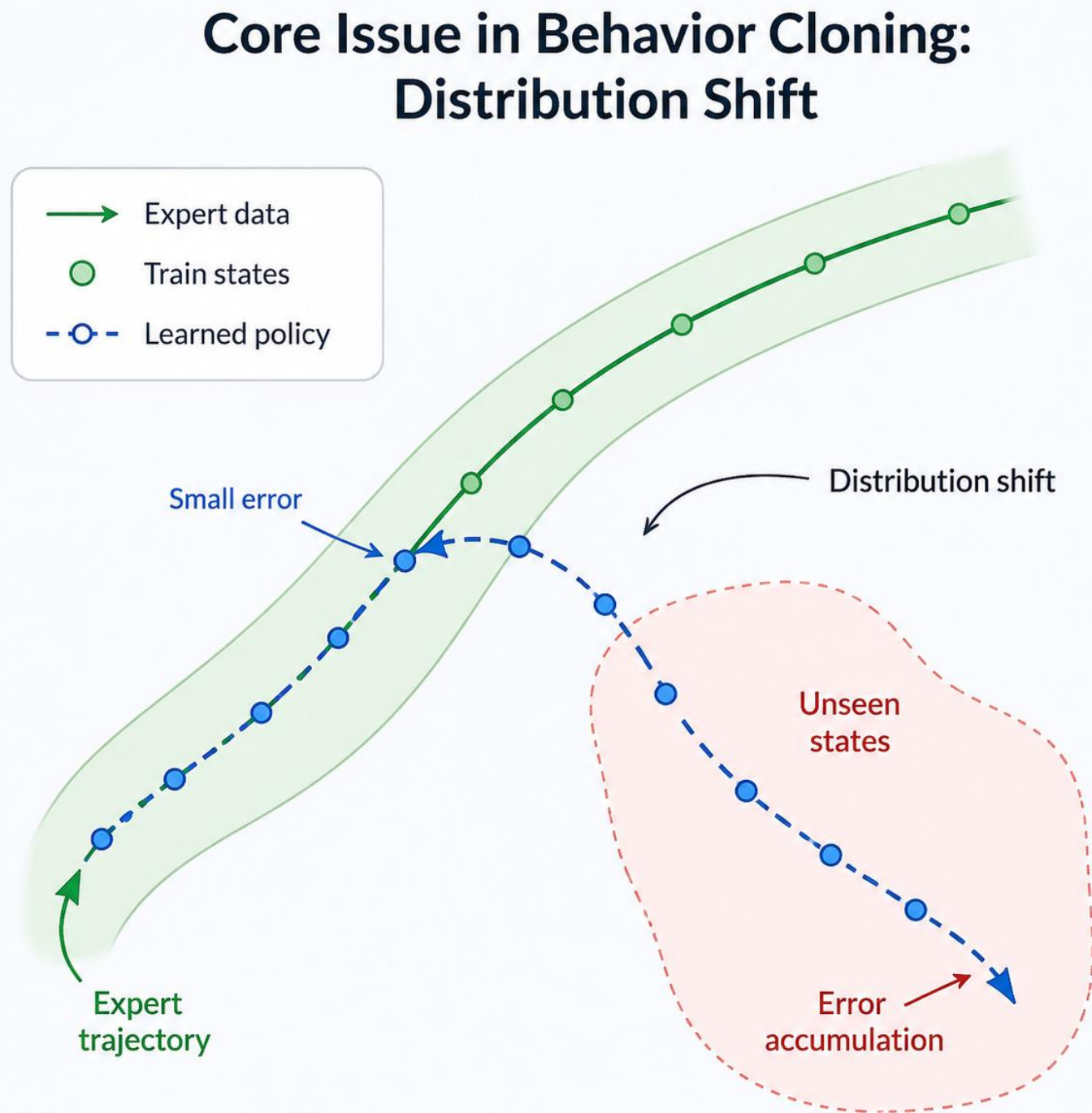


Figure: Distribution Shift

01. Root Cause: Train-Test Distribution Gap

- ✓ Training: Learns from expert's perfect states.
- ✓ Deployment: Faces its own real states.

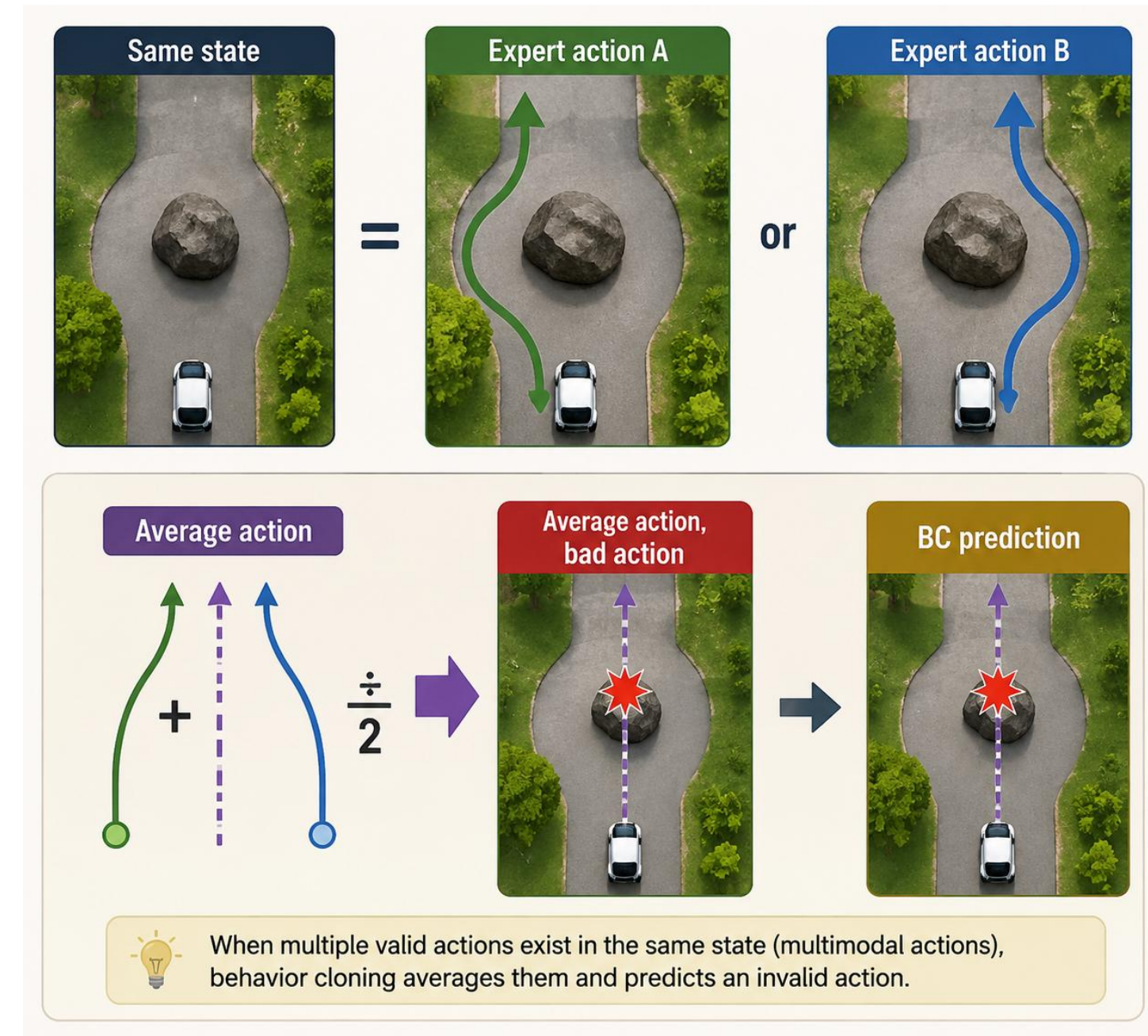
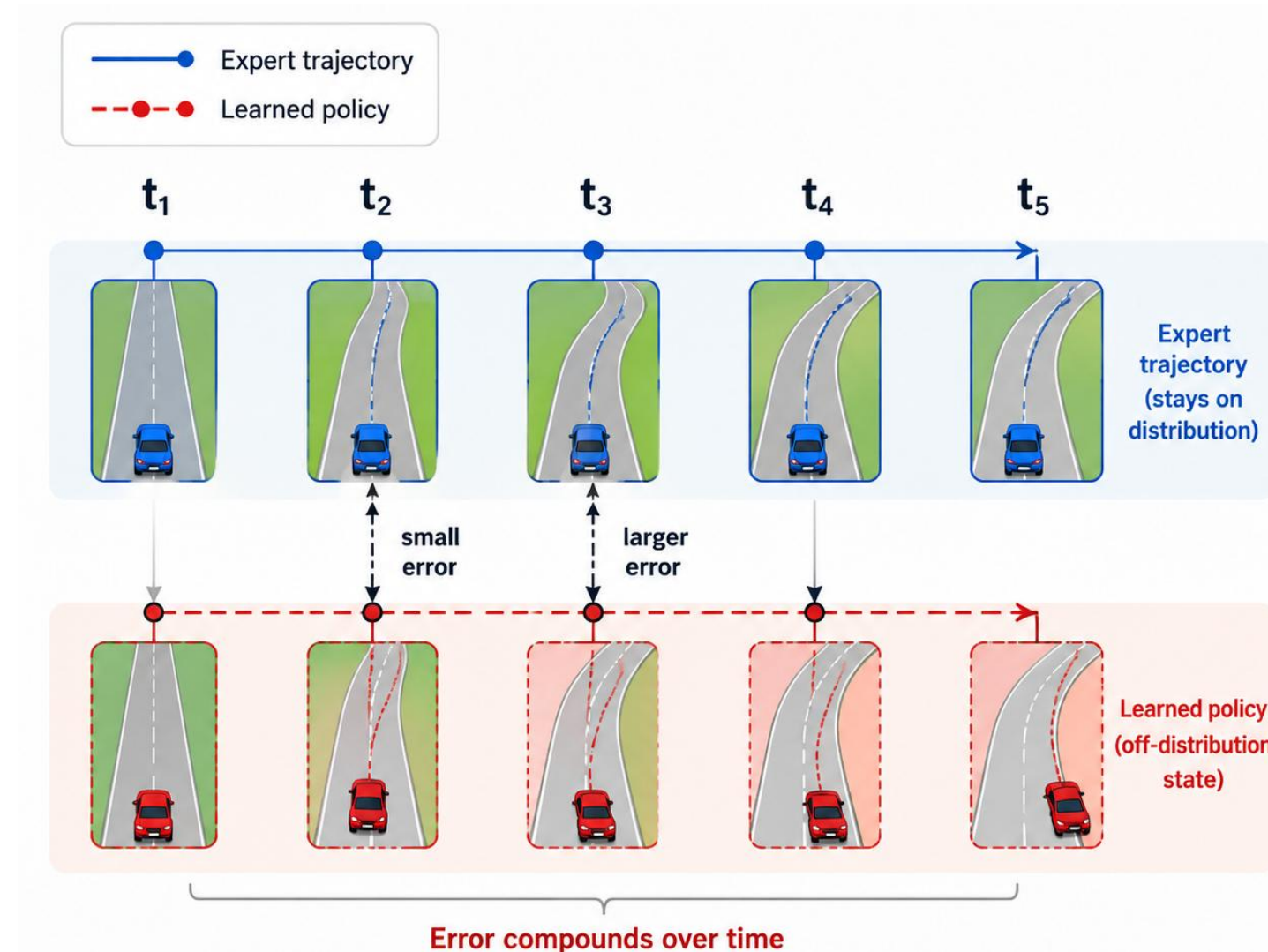
02. Cascading Shift Mechanism

- ① Tiny Gap: Slight policy difference creates initial deviation.
- ② Unknown States: Robot enters states unseen in training.
- ③ Error Amplification: Lack of data support causes exponential error growth.
- ④ Completely diverges from the reference trajectory.

03. Consequence: Rapid Task Failure

- ✓ Strategy quickly enters Out-of-Distribution (OOD) zones.
- ✓ No expert data to correct errors → Task fails rapidly.

6.2 Core Issues in BC: Compounding Error and Multimodal Actions



01 Compounding Error

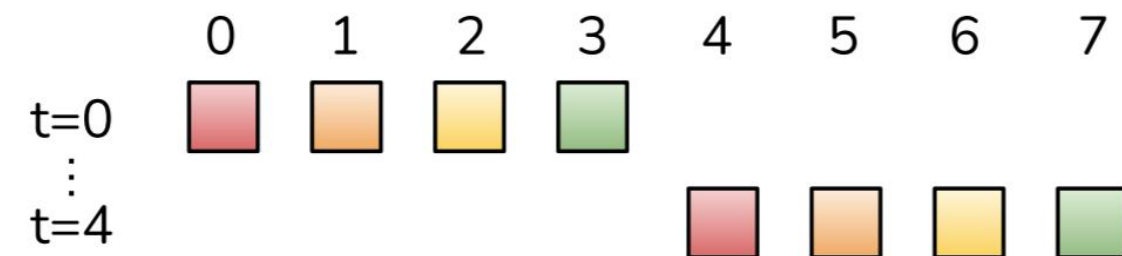
- **Mechanism:** Tiny prediction errors accumulate over long sequences.
- **Consequence:** Like a snowball, small deviations amplify exponentially, causing the system to completely diverge from the expected trajectory.

02 Multimodal Actions

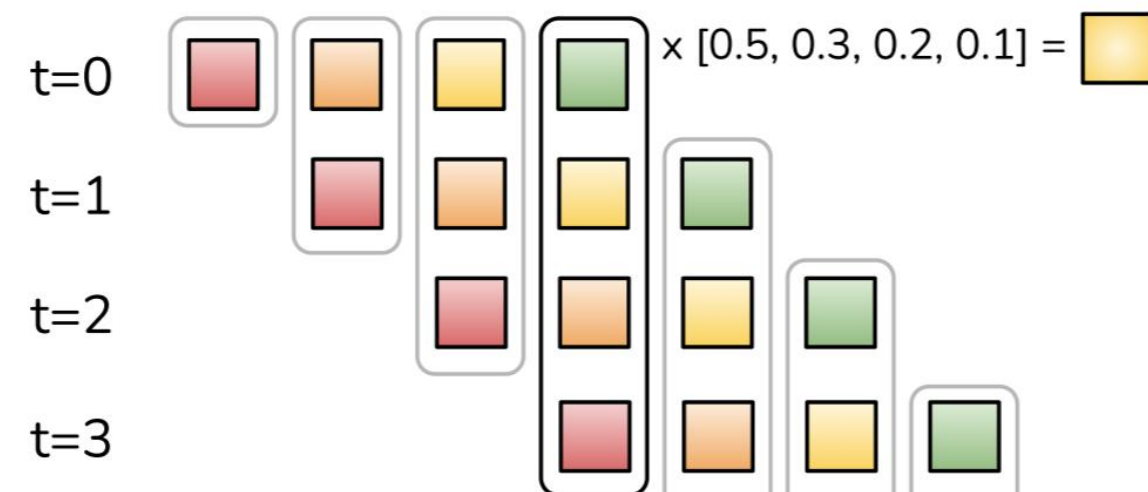
- **Mechanism:** Multiple valid paths exist for the same observation (e.g., going left or right around an obstacle).
- **Consequence:** Standard MSE regression loss fails to model this multimodal distribution, leading to "averaged-out" and often invalid actions.

7. Action Chunking

Action Chunking



Action Chunking + Temporal Ensemble



01 Issues with Single-Step Action Prediction

- ✓ **Frame-by-Frame:** Independent predictions can cause action jitter.
- ✓ **Error Accumulation:** High-frequency control amplifies errors over time.
- ✓ **Short-Sighted:** Lacks short-term intent (only predicts the next step).
- ✓ **Coordination Issues:** Unstable for dual-arm & high-DoF tasks.

02 Core Idea: Sequence Generation

$$\pi_{\theta}(o_t) \rightarrow a_t, a_{t+1}, \dots, a_{t+k-1}$$

Or:

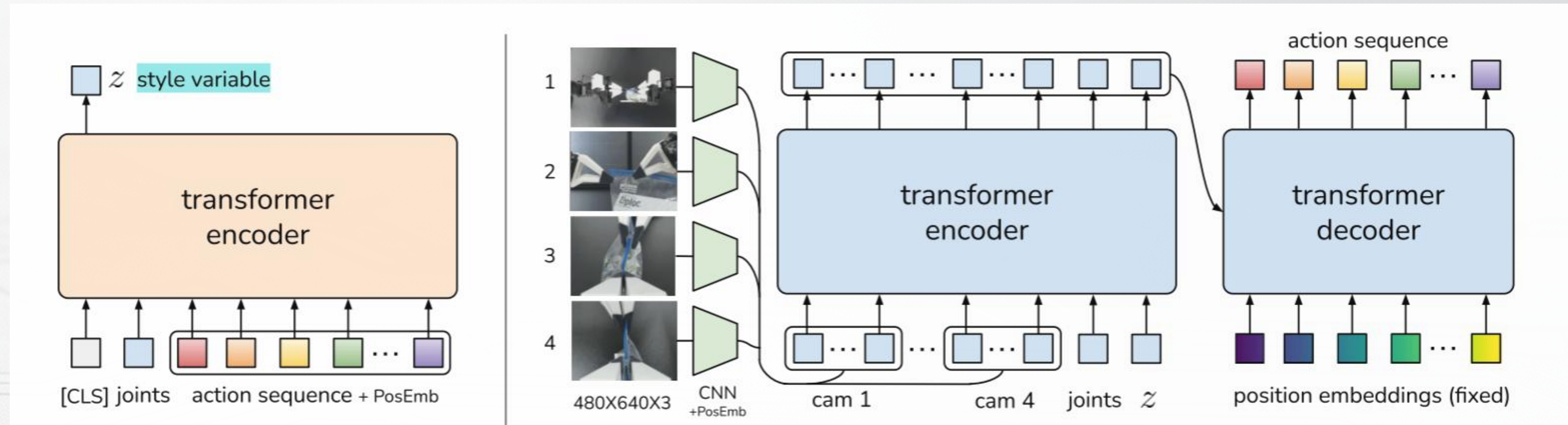
$$\pi_{\theta}(o_t) \rightarrow A_t$$

$$A_t = [a_t, a_{t+1}, \dots, a_{t+k-1}]$$

03 Action Horizon vs. Execution Horizon

- **Action Horizon:** Number of future steps predicted by the model.
- **Execution Horizon:** Number of steps actually executed.

8. ACT: Action Chunking with Transformers



Transformer Architecture

Processes multi-view images and proprioceptive states to efficiently model complex temporal dependencies.



Action Chunking

Predicts multiple consecutive future actions in a single pass, ensuring smooth and coherent manipulation.



CVAE Diversity Modeling

Introduces Conditional Variational Autoencoders to generate diverse action distributions.



Temporal Ensembling

Fuses predictions from adjacent time steps to effectively suppress jitter and yield smoother motions.

Thanks!

Presenter: Mengyuan Wang

Date: 2026/08/15

Next: From VLM to VLA.

